

UNIVERSIDAD AUTÓNOMA DE ZACATECAS

UNIDAD ACADÉMICA DE MATEMÁTICAS

LICENCIATURA EN ACTUARÍA

Grandes Bases de datos



Actividad 2: Clustering Jerárquico

Presenta:

Brianna Camila Meza Dueñez

Profesor:

Dr. José Raúl Castro Esparza

Zacatecas, Zacatecas, a 31 de Agosto de 2026



Objetivo:

El objetivo de esta actividad es que realices un análisis de agrupación por “Clustering jerárquico” a una base de datos de manera que puedas generar conclusiones de acuerdo a diversas categorías.

Pasos a seguir:

Como primeros pasos, descargamos el excel de amazon para poder agregarlo en nuestro escritorio y poder empezar a trabajarlo.

Agregamos el título de “Clientes” a la primera columna, dejamos solo las columnas numéricas para poder hacer los cálculos correspondientes. Nuestra columna de “Clientes” la guardamos para después poder identificarlos.

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
```

```
[2]: data = pd.read_excel(r"C:\Users\brian\OneDrive\Desktop\Amazon.xlsx")
data.head()
```

```
[2]:
```

	Unnamed: 0	Velocidad Entrega	Precio	Durabilidad	Imagen Producto	Valor Educativo	Servicio Retorno	Tamano Paquete	Calidad Producto	Numero Estrellas
0	Adam	205	3	345	235	24	23	26	21	17
1	Anna	9	15	315	33	25	4	42	215	28
2	Bernard	17	26	285	3	43	27	41	26	33
3	Edward	135	5	355	295	18	23	39	195	17
4	Emilia	3	45	48	39	34	46	225	34	43

```
[7]: #Vamos a dejar solo las columnas numericas y guardar los nombres de los clientes aparte, para despues identificarlos
data = data.rename(columns = {'Unnamed: 0': 'Cliente'})
data.head()
nombres = data['Cliente']
X = data.drop(columns = ['Cliente'])
```

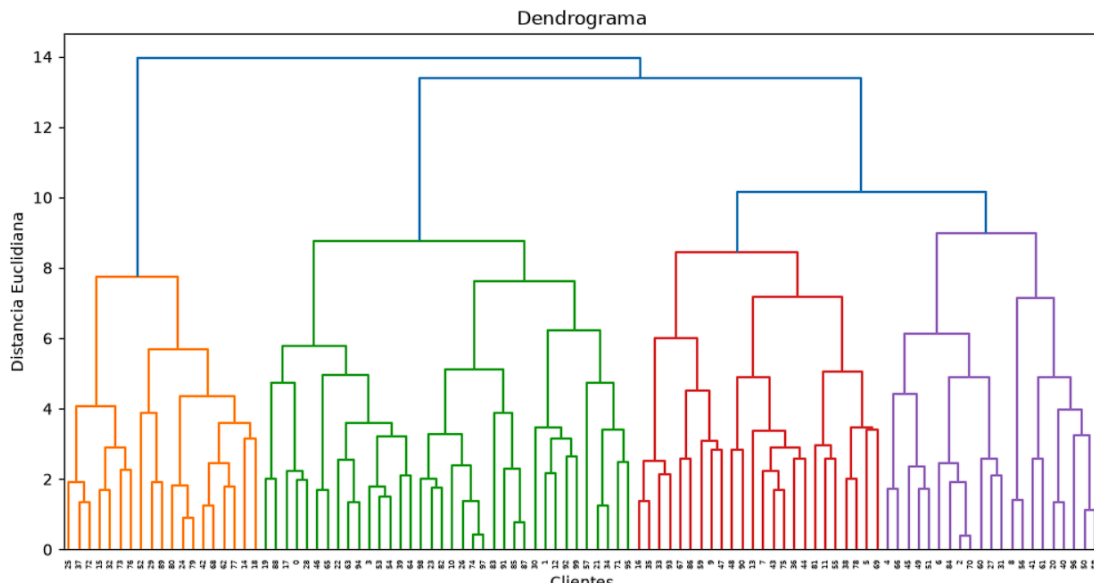
Estandarizamos nuestros datos, para poder trabajar bajo la misma escala de datos.

```
[9]: #Vamos a estandarizar Los datos
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
data_scale = scaler.fit_transform(X)
data_scale

[9]: array([[ 1.20323754, -0.85570178,  0.57137311,  0.75217596, -0.13889518,
           -0.08973907, -0.79595768, -0.98297775, -0.96213678],
          [-0.86985105, -0.67018378,  0.41102518, -0.91510753, -0.04985981,
           -2.07234645, -0.69782003,  0.77947773, -0.01886543],
          [-0.78523519, -0.50012561,  0.25067726, -1.16272389,  1.55277686,
           0.32765196, -0.70395363, -0.93755364,  0.40989428],
          [ 0.46284876, -0.82478211,  0.62482242,  1.24740868, -0.6731074 ,
           -0.08973907, -0.71622084,  0.59778129, -0.96213678],
          [-0.93331294, -0.20638878, -1.01607136, -0.86558426,  0.75145853,
           2.31025933,  0.42462935, -0.86487506,  1.26741369],
          [ 0.03976945,  1.64879124,  0.83861965, -0.98939243,  0.03917556,
           -0.5071301 ,  2.01936616, -0.97389292, -2.16266395],
          [-0.72177329, -0.71656328,  1.26621412, -0.91510753,  0.8404939 ,
           2.20591158, -0.72235444, -0.91029917,  0.2383904 ],
          [-0.27754002, -0.57742478, -1.10693518,  0.91725354,  0.2172463 ,
           -0.19408683,  1.16066172,  0.77947773,  0.06688651],
          [ 1.94362632, -0.77840261, -1.02141629,  0.75217596,  0.8404939 ,
           -2.1766942 , -0.72235444, -0.92846881,  1.69617339],
          [-0.94388993,  1.80338957,  0.46447449, -1.16272389,  1.01856464,
           0.84939074,  1.71268601, -0.92846881,  0.92440592],
          [-0.8381201 , -0.77840261, -1.03745108, -0.98939243, -2.09767333,
           0.43199971, -0.77755687,  0.77947773, -0.96213678],
          [ 1.09746771, -0.73202311,  1.15931551, -0.99764631, -2.00863795,
           0.11895644,  1.59001394, -0.94663846,  0.32414234],
          [-0.81696613, -0.79386245,  0.89206896, -1.03066183, -0.40600129,
           -1.02886888, -0.75302246, -0.97389292, -0.36187319],
          [ 0.99169789,  0.25740623, -1.04279601,  1.1648699 ,  0.12821093,
           1.37112952,  1.09932569, -0.94663846, -0.44762513],
          [ 1.52054702,  0.10280789,  1.37311274,  1.57756383, -2.00863795,
```

Después creamos el dendograma correspondiente para poder decidir cuántos clusters utilizaremos en esta actividad.

```
plt.title('Dendrograma')
plt.xlabel('Clientes')
plt.ylabel('Distancia Euclidiana')
plt.show()
```



Hacemos códigos para poder saber cuantos colores de clusters existen, cuales son los colores únicos y cuáles serían los colores óptimos de clusters.

[illegible]

Creamos un código para poder hacer el clustering jerárquico, esto para después poder crear una tabla con los nombres de los clientes y en que cluster se encuentran ellos.

```
[17]: #Vamos hacer el clustering jerarquico
from sklearn.cluster import AgglomerativeClustering
cluster = AgglomerativeClustering(n_clusters=4, linkage='ward')
grupos = cluster.fit_predict(data_scale)

[18]: resultado = pd.DataFrame({'Cliente': nombres, 'Cluster': grupos})
resultado.sort_values('Cluster')
```

	Cliente	Cluster
2	Bernard	0
4	Emilia	0
6	Philip	0
8	Xavier	0
27	Florence	0
...	...	...
72	Methodius	3
89	Stephania	3
80	Myroslava	3
68	Maura	3
73	Michael	3

100 rows × 2 columns

En esta actividad nos pide encontrar: ¿qué productos recomendaría a Salomé? ¿Qué productos recomendaría a Stephanía? ¿Qué productos recomendaría a Lydia?

Para poder contestar esas preguntas creamos los siguientes códigos para poder encontrar a las personas que buscamos. Con el código anterior pudimos localizar primero a Stephania que se encontraba en el cluster 3, usamos un código para poder ver quien mas estaba en ese cluster.

```
[27]: #Vamos a encontrar quien mas esta en el cluster 3, ya que Stephania salio en ese cluster
resultado[resultado['Cluster'] == 3]
```

	Cliente	Cluster
14	Isidore	3
15	Joseph	3
18	Eunice	3
24	Flavia	3
25	Flora	3
29	Helen	3
32	Lourdes	3
37	Jervis	3
42	Judith	3
52	Louise	3
62	Markian	3
68	Maura	3
72	Methodius	3
73	Michael	3
76	Monica	3

Se hizo el siguiente código para poder ver las calificaciones de los clientes que pertenecen al cluster 3, con esto hicimos otro código para poder encontrar quien era similar a Stephanie.

```
[28]: # Ver las calificaciones de todo el clúster 3, con nombres
data_con_cluster = data.copy()
data_con_cluster['Cluster'] = grupos

cluster3 = data_con_cluster[data_con_cluster['Cluster'] == 3]
cluster3
```

[28]:

	Cliente	Velocidad Entrega	Precio	Durabilidad	Imagen Producto	Valor Educativo	Servicio Retorno	Tamaño Paquete	Calidad Producto	Número Estrellas	Cluster
14	Isidore	235	65	495	335	3	26	34	295	39	3
15	Joseph	17	1	485	235	27	17	24	235	34	3
18	Eunice	265	7	485	305	33	39	34	295	39	3
24	Flavia	255	7	435	24	33	26	19	245	34	3
25	Flora	23	105	395	29	34	28	235	295	34	3
29	Helen	205	185	295	275	39	3	42	255	31	3
32	Lourdes	26	1	465	295	37	24	23	305	45	3
37	Jervis	2	45	455	27	24	26	365	255	31	3
42	Judith	245	115	465	225	36	13	31	295	38	3
52	Louise	225	205	315	295	43	34	44	275	35	3
62	Markian	205	55	465	275	25	27	37	265	32	3
68	Maura	265	85	425	185	35	19	24	215	43	3
72	Methodius	18	11	495	24	29	19	245	245	36	3
73	Michael	26	65	455	225	33	27	365	255	45	3
76	Manies	10	4	415	205	22	26	265	255	27	3

```
#Vamos a encontrar el cliente que sea similar a Stephanie dentro del cluster 3
from scipy.spatial.distance import euclidean

# Filtra solo el clúster de Stephanie (ajusta el número si es necesario)
cluster_stephanie = data_con_cluster[data_con_cluster['Cluster'] == 3].copy()

# Guarda los datos numéricos de Stephanie (sin nombre ni columna Cluster)
fila_stephanie = cluster_stephanie[cluster_stephanie['Cliente'] == 'Stephanie'].drop(columns=['Cliente', 'Cluster']).values[0]

# Calcula la distancia de Stephanie contra cada persona de su clúster
distancias = {}
for i, row in cluster_stephanie.iterrows():
    nombre = row['Cliente']
    if nombre != 'Stephanie':
        valores = row.drop(labels=['Cliente', 'Cluster']).values
        distancias[nombre] = euclidean(fila_stephanie, valores)

# Ordena de menor a mayor distancia (el más parecido primero)
distancias_ordenadas = sorted(distancias.items(), key=lambda x: x[1])
distancias_ordenadas
```

```
[('Isidore', np.float64(83.00602387778854)),
 ('Markian', np.float64(94.75758544834287)),
 ('Judith', np.float64(96.79876032264049)),
 ('Eunice', np.float64(135.2590107904091)),
 ('Maura', np.float64(176.34908562280668)),
 ('Louise', np.float64(177.28508115461943)),
 ('Helen', np.float64(192.21602430598756)),
 ('Mykyta', np.float64(202.18803129760178)),
 ('Lourdes', np.float64(228.25862524776582)).
```

Encontramos que el más similar a Stephanie es Isidore.

Ahora para encontrar en que cluster y quiénes son similares a Lydia y a Salome, creamos el siguiente código para poder hacerlo en una sola línea y poder encontrarlas.

```
[30]: #Vamos a encontrar a Lydia, su cluster y cual es el cliente que es mas similar a ella
from scipy.spatial.distance import euclidean

def cliente_mas_similar(nombre_cliente, data_con_cluster):
    # Encuentra el clúster del cliente
    cluster_num = data_con_cluster[data_con_cluster['Cliente'] == nombre_cliente]['Cluster'].values[0]

    # Filtra a todos los del mismo clúster
    mismo_cluster = data_con_cluster[data_con_cluster['Cluster'] == cluster_num].copy()

    # Datos numéricos del cliente en cuestión
    fila_cliente = mismo_cluster[mismo_cluster['Cliente'] == nombre_cliente].drop(columns=['Cliente', 'Cluster']).values[0]

    # Calcula distancias contra el resto del clúster
    distancias = {}
    for i, row in mismo_cluster.iterrows():
        nombre = row['Cliente']
        if nombre != nombre_cliente:
            valores = row.drop(labels=['Cliente', 'Cluster']).values
            distancias[nombre] = euclidean(fila_cliente, valores)

    distancias_ordenadas = sorted(distancias.items(), key=lambda x: x[1])

    print(f"{nombre_cliente} pertenece al clúster {cluster_num}")
    return distancias_ordenadas

[31]: cliente_mas_similar('Lydia', data_con_cluster)

Lydia pertenece al clúster 2

[31]: [('Fedir', np.float64(41.569219381653056)),
      ('Ivan', np.float64(141.16656828017037)),
      ('Lubomyr', np.float64(143.41548033598048)),
      ('Anna', np.float64(169.25424662323837)),
```

```
[32]: cliente_mas_similar('Salome', data_con_cluster)
```

Salome pertenece al clúster 1

```
[32]: [('Magdalyna', np.float64(184.95675170158023)),
      ('Kalyna', np.float64(192.47337478207214)),
      ('Gabriel', np.float64(207.25829295832773)),
      ('Joachim', np.float64(239.98958310726738)),
      ('Fabian', np.float64(267.99813432186426)),
      ('Henry', np.float64(273.7206605282108)),
      ('Maya', np.float64(275.1999273255718)),
      ('Jeremiah', np.float64(291.6796187600361)),
      ('Myron', np.float64(311.61996084975044)),
      ('Mina', np.float64(326.98317999554655)),
      ('Justin', np.float64(333.6090526349667)),
      ('Leonard', np.float64(368.55800086282215)),
      ('Jacob', np.float64(376.58730727415656)),
      ('Frank', np.float64(380.2341383937008)),
      ('Isabelle', np.float64(381.255819627714)),
      ('Leon', np.float64(393.3090387977373)),
      ('Susanna', np.float64(401.0660793435416)),
      ('Eugene', np.float64(401.3016321920458)),
      ('Maria', np.float64(420.984560286954)),
      ('Matthew', np.float64(436.9004463261625)),
      ('Sebastian', np.float64(442.6703062099377)),
      ('Tamara', np.float64(445.7869446271391)),
      ('Isadore', np.float64(469.5742752749558))]
```

Respuestas de las preguntas:

¿Qué productos recomendaría a Salomé?

Recomendaría a Salome los mismos productos que compro Magdalyna

¿Qué productos recomendaría a Stephanía?

Recomendaría a Salome los mismos productos que compro Isidore

¿Qué productos recomendaría a Lydia?

Recomendaría a Salome los mismos productos que compro Fedir